



Green University of Bangladesh

*Department of Computer Science and Engineering (CSE)
Semester: (Fall, Year: 2024), B.Sc. in CSE (Day)*

E – learningQuizPlatform

*Course Title: Database Lab
Course Code: CSE 210
Section: 231 D2*

Students Details

Name	ID
MD MAHABUB HASAN MAHIN	231902056

*Submission Date: 25-11-2024
Course Teacher's Name: Umme Habiba*

[For teachers use only: **Don't write anything inside this box**]

<u>Lab Project Status</u>	
Marks:	Signature:
Comments:	Date:

Contents

1	Introduction	2
1.1	Overview	2
1.2	Motivation	2
1.2.1	Problem Statement	2
1.3	Design Goals/Objectives	2
1.4	Technologies Used (Database and Java UI Design)	3
1.5	Features	3
2	Implementation of the Project	4
2.1	Introduction	4
2.2	Design	4
2.3	Implementation of the E-learning Quiz Platform	4
3	Performance Evaluation	40
3.1	Results Analysis	40
4	Conclusion	44
4.1	Discussion	44
4.2	Limitations	44
4.3	Scope of Future Work	44

Chapter 1

Introduction

1.1 Overview

The E-Learning Quiz Platform is an interactive system designed for educational purposes, allowing users to take quizzes, track performance, and manage quiz content. It leverages Java Swing for a user-friendly desktop interface and MySQL for efficient data storage and retrieval.

1.2 Motivation

Traditional quiz management systems lack automation, scalability, and real-time feedback, making quiz creation, grading, and performance tracking inefficient for both educators and students.

1.2.1 Problem Statement

The problem this project addresses is the inefficiency and limitations of current e-learning platforms or manual quiz systems in educational settings. Specifically, the lack of an automated system to create, take, and grade quizzes, combined with inadequate tracking of student progress, results in inefficiencies and missed learning opportunities. The goal is to create a system that automates the entire process, provides real-time feedback to students, allows for easy quiz creation and management, and tracks the progress and performance of students.

1.3 Design Goals/Objectives

Efficient Quiz Management: Enable admins to create, edit, and delete quizzes easily.

User-Friendly Interface: Provide an intuitive GUI using Java Swing for seamless navigation.

Real-Time Feedback: Provide instant feedback to students after quiz submission.

Data Integrity: Ensure secure and accurate data handling using MySQL / Microsoft

SQL server.

Scalability: Design the system to handle increasing numbers of users and quizzes.

Performance Tracking: Track and display user progress and results over time.

1.4 Technologies Used (Database and Java UI Design)

1. Database Management

- Microsoft SQL server /MySQL
- JDBC (Java Database Connectivity)

2. Java UI Design

- JavaFX
- Swing
- Scene Builder

3. Backend Development

- Java (Spring Boot)

4. Security

- JDBC Authentication
- AES Encryption

1.5 Features

- **User Authentication:** Admin and student login with role-based access.
- **Quiz Creation:** Admins can create quizzes, add questions, and set correct answers.
- **Taking Quizzes:** Students can take quizzes, select answers, and submit for grading.
- **Instant Results:** Immediate feedback on quiz scores and correct answers.
- **Progress Tracking:** Students can view their quiz history and results.
- **Database Integration:** Data for quizzes, questions, users, and results stored in MySQL/Microsoft SQL server.
- **Admin Dashboard:** Admins can view user statistics, manage quizzes, and edit content.

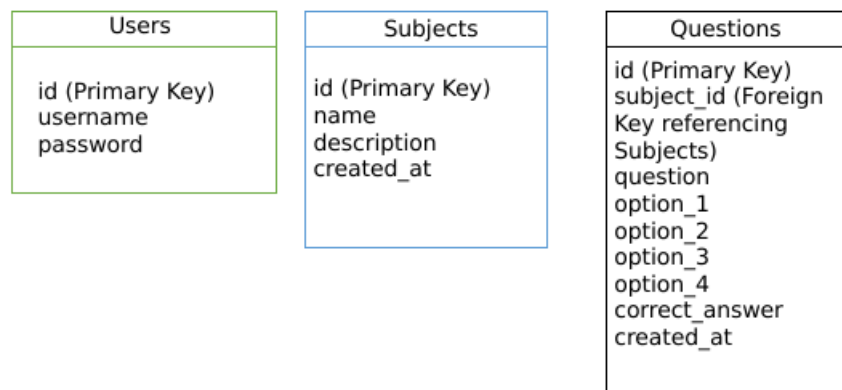
Chapter 2

Implementation of the Project

2.1 Introduction

This project, **E-Learning Quiz Platform**, is designed to revolutionize the learning experience by integrating a dynamic quiz system. The platform aims to cater to a diverse audience, including students, educators, and professionals, by providing an engaging and user-friendly environment to assess and enhance their knowledge.

2.2 Design



2.3 Implementation of the E-learning Quiz Platform

Project Setup

```
1 -- Users Table
2 CREATE TABLE users (
3     id INT AUTO_INCREMENT PRIMARY KEY,
```

```

4      username VARCHAR(255) NOT NULL UNIQUE,
5      password VARCHAR(255) NOT NULL
6  );
7
8  -- Subjects Table
9  CREATE TABLE subjects (
10     id INT AUTO_INCREMENT PRIMARY KEY,
11     name VARCHAR(255) NOT NULL UNIQUE,
12     description TEXT,
13     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
14 );
15
16 -- Questions Table
17 CREATE TABLE questions (
18     id INT AUTO_INCREMENT PRIMARY KEY,
19     subject_id INT NOT NULL, -- Added to associate questions
20     with a specific subject
21     question TEXT NOT NULL,
22     option_1 VARCHAR(255) NOT NULL,
23     option_2 VARCHAR(255) NOT NULL,
24     option_3 VARCHAR(255),
25     option_4 VARCHAR(255),
26     correct_answer INT NOT NULL CHECK (correct_answer BETWEEN 1
27     AND 4),
28     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
29     FOREIGN KEY (subject_id) REFERENCES subjects(id) ON DELETE
30     CASCADE
31 );

```

Listing 2.1: Database code

Java Code: E-learning Quiz Platform

```

1  package quizzes;
2
3  import java.sql.Connection;
4  import java.sql.DriverManager;
5  import java.sql.SQLException;
6
7  public class Quiz_Platform {
8
9      public static void main(String[] args) {
10
11          login n =new login();
12          n.setVisible(true);
13      }
14  }

```

Listing 2.2: Quiz Platform

```

1  package quizzes;
2  import java.security.MessageDigest;
3  import quizzes.Users;
4  import java.sql.Connection;
5  import java.sql.DriverManager;

```

```

6 import java.sql.PreparedStatement;
7 import java.sql.ResultSet;
8 import java.util.logging.Level;
9 import java.util.logging.Logger;
10 import javax.swing.JOptionPane;
11
12 public class login extends javax.swing.JFrame {
13     private String userName;
14     private String passWord;
15     public login() {
16         initComponents();
17     }
18     private Connection connectToDatabase() {
19         Connection conn = null;
20         try {
21             String url = "jdbc:mysql://localhost:3306/last?
                zeroDateTimeBehavior=CONVERT_TO_NULL ";
22             String username = "root";
23             String password = "";
24             conn = DriverManager.getConnection(url, username,
                password);
25         } catch (Exception e) {
26             JOptionPane.showMessageDialog(this, "Database
                connection failed: " + e.getMessage(), "Error",
                JOptionPane.ERROR_MESSAGE);
27         }
28         return conn;
29     }
30     @SuppressWarnings("unchecked")
31     // <editor-fold defaultstate="collapsed" desc="Generated
        Code">
32     private void initComponents() {
33
34         jPanel1 = new javax.swing.JPanel();
35         jPanel2 = new javax.swing.JPanel();
36         jPanel3 = new javax.swing.JPanel();
37         jLabel3 = new javax.swing.JLabel();
38         jPanel4 = new javax.swing.JPanel();
39         Login = new javax.swing.JButton();
40         newUser = new javax.swing.JButton();
41         jPanel6 = new javax.swing.JPanel();
42         txtName = new javax.swing.JTextField();
43         user = new javax.swing.JLabel();
44         jPanel7 = new javax.swing.JPanel();
45         Password = new javax.swing.JLabel();
46         txtpassword = new javax.swing.JPasswordField();
47         Add_Question = new javax.swing.JButton();
48         jPanel5 = new javax.swing.JPanel();
49         jLabel4 = new javax.swing.JLabel();
50
51         setDefaultCloseOperation(javax.swing.WindowConstants.
            EXIT_ON_CLOSE);
52
53         jPanel1.setLayout(new java.awt.BorderLayout());
54
55         jPanel3.setBorder(javax.swing.BorderFactory.
            createLineBorder(new java.awt.Color(255, 255, 255)))
            ;

```

```

56
57     jLabel3.setText("Welcome Online quiz platform");
58
59     javax.swing.GroupLayout jPanel3Layout = new javax.swing
        .GroupLayout(jPanel3);
60     jPanel3.setLayout(jPanel3Layout);
61     jPanel3Layout.setHorizontalGroup(
62         jPanel3Layout.createParallelGroup(javax.swing.
            GroupLayout.Alignment.LEADING)
63         .addGroup(jPanel3Layout.createSequentialGroup()
64             .addGap(93, 93, 93)
65             .addComponent(jLabel3)
66             .addGap(108, Short.MAX_VALUE))
67     );
68     jPanel3Layout.setVerticalGroup(
69         jPanel3Layout.createParallelGroup(javax.swing.
            GroupLayout.Alignment.LEADING)
70         .addGroup(jPanel3Layout.createSequentialGroup()
71             .addGap(93, 93, 93)
72             .addComponent(jLabel3)
73             .addGap(108, Short.MAX_VALUE))
74     );
75
76     jPanel4.setBorder(javax.swing.BorderFactory.
        createBevelBorder(javax.swing.border.BevelBorder.
            RAISED));
77
78     Login.setText("Login");
79     Login.addActionListener(new java.awt.event.
        ActionListener() {
80         public void actionPerformed(java.awt.event.
            ActionEvent evt) {
81             LoginActionPerformed(evt);
82         }
83     });
84
85     newuser.setText("create user");
86     newuser.addActionListener(new java.awt.event.
        ActionListener() {
87         public void actionPerformed(java.awt.event.
            ActionEvent evt) {
88             newuserActionPerformed(evt);
89         }
90     });
91
92     txtName.setFont(new java.awt.Font("Liberation Sans", 0,
        18)); // NOI18N
93     txtName.setBorder(javax.swing.BorderFactory.
        createEtchedBorder());
94     txtName.addActionListener(new java.awt.event.
        ActionListener() {
95         public void actionPerformed(java.awt.event.
            ActionEvent evt) {
96             txtNameActionPerformed(evt);
97         }
98     });
99
100    user.setText("User Name ");

```

```

101
102 javax.swing.GroupLayout jPanel6Layout = new javax.swing
    .GroupLayout(jPanel6);
103 jPanel6.setLayout(jPanel6Layout);
104 jPanel6Layout.setHorizontalGroup(
105     jPanel6Layout.createParallelGroup(javax.swing.
        GroupLayout.Alignment.LEADING)
106     .addGroup(jPanel6Layout.createSequentialGroup()
107         .addGap(105, 105, 105)
108         .addGroup(jPanel6Layout.createParallelGroup(
            javax.swing.GroupLayout.Alignment.LEADING)
109             .addGroup(jPanel6Layout.
                createSequentialGroup()
110                 .addComponent(user)
111                 .addGap(105, 105, 105))
112             .addGroup(jPanel6Layout.
                createSequentialGroup()
113                 .addComponent(txtName, javax.swing.
                    GroupLayout.DEFAULT_SIZE, 187, Short
                    .MAX_VALUE)
114                 .addGap(105, 105, 105)))
115     );
116 jPanel6Layout.setVerticalGroup(
117     jPanel6Layout.createParallelGroup(javax.swing.
        GroupLayout.Alignment.LEADING)
118     .addGroup(jPanel6Layout.createSequentialGroup()
119         .addGap(105, 105, 105)
120         .addComponent(user)
121         .addPreferredGap(javax.swing.LayoutStyle.
            ComponentPlacement.RELATED, 12, Short.
            MAX_VALUE)
122         .addComponent(txtName, javax.swing.GroupLayout.
            PREFERRED_SIZE, 22, javax.swing.GroupLayout.
            PREFERRED_SIZE)
123         .addGap(105, 105, 105))
124     );
125
126 Password.setText("Password");
127
128 txtpassword.addActionListener(new java.awt.event.
    ActionListener() {
129     public void actionPerformed(java.awt.event.
       (ActionEvent evt) {
130         txtpasswordActionPerformed(evt);
131     }
132 });
133
134 javax.swing.GroupLayout jPanel7Layout = new javax.swing
    .GroupLayout(jPanel7);
135 jPanel7.setLayout(jPanel7Layout);
136 jPanel7Layout.setHorizontalGroup(
137     jPanel7Layout.createParallelGroup(javax.swing.
        GroupLayout.Alignment.LEADING)
138     .addGroup(jPanel7Layout.createSequentialGroup()
139         .addGap(105, 105, 105)
140         .addGroup(jPanel7Layout.createParallelGroup(
            javax.swing.GroupLayout.Alignment.LEADING)
141             .addComponent(Password)

```

```

142         .addComponent(txtpassword, javax.swing.
           GroupLayout.PREFERRED_SIZE, 186, javax.
           swing.GroupLayout.PREFERRED_SIZE))
143         .addContainerGap())
144     );
145     JPanel7Layout.setVerticalGroup(
146         JPanel7Layout.createParallelGroup(javax.swing.
           GroupLayout.Alignment.LEADING)
147         .addGroup(JPanel7Layout.createSequentialGroup())
148         .addContainerGap()
149         .addComponent>Password)
150         .addPreferredGap(javax.swing.LayoutStyle.
           ComponentPlacement.RELATED)
151         .addComponent(txtpassword)
152         .addContainerGap())
153     );
154
155     Add_Question.setText("Add_Question");
156     Add_Question.addActionListener(new java.awt.event.
       ActionListener() {
157         public void actionPerformed(java.awt.event.
           ActionEvent evt) {
158             Add_QuestionActionPerformed(evt);
159         }
160     });
161
162     javax.swing.GroupLayout JPanel4Layout = new javax.swing
       .GroupLayout(JPanel4);
163     JPanel4.setLayout(JPanel4Layout);
164     JPanel4Layout.setHorizontalGroup(
165         JPanel4Layout.createParallelGroup(javax.swing.
           GroupLayout.Alignment.LEADING)
166         .addGroup(JPanel4Layout.createSequentialGroup())
167         .addGap(32, 32, 32)
168         .addGroup(JPanel4Layout.createParallelGroup(
           javax.swing.GroupLayout.Alignment.LEADING)
169         .addComponent(JPanel7, javax.swing.
           GroupLayout.PREFERRED_SIZE, javax.swing.
           GroupLayout.DEFAULT_SIZE, javax.swing.
           GroupLayout.PREFERRED_SIZE)
170         .addComponent(JPanel6, javax.swing.
           GroupLayout.PREFERRED_SIZE, javax.swing.
           GroupLayout.DEFAULT_SIZE, javax.swing.
           GroupLayout.PREFERRED_SIZE)
171         .addGroup(JPanel4Layout.
           createSequentialGroup()
172         .addComponent(Login)
173         .addGap(18, 18, 18)
174         .addComponent(newuser)
175         .addPreferredGap(javax.swing.
           LayoutStyle.ComponentPlacement.
           UNRELATED)
176         .addComponent(Add_Question)))
177         .addContainerGap(22, Short.MAX_VALUE))
178     );
179     JPanel4Layout.setVerticalGroup(
180         JPanel4Layout.createParallelGroup(javax.swing.
           GroupLayout.Alignment.LEADING)

```

```

181         .addGroup(jPanel4Layout.createSequentialGroup()
182             .addContainerGap()
183             .addComponent(jPanel6, javax.swing.GroupLayout.
                PREFERRED_SIZE, javax.swing.GroupLayout.
                DEFAULT_SIZE, javax.swing.GroupLayout.
                PREFERRED_SIZE)
184             .addPreferredGap(javax.swing.LayoutStyle.
                ComponentPlacement.RELATED)
185             .addComponent(jPanel7, javax.swing.GroupLayout.
                PREFERRED_SIZE, javax.swing.GroupLayout.
                DEFAULT_SIZE, javax.swing.GroupLayout.
                PREFERRED_SIZE)
186             .addGap(18, 18, 18)
187             .addGroup(jPanel4Layout.createParallelGroup(
                javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(Login)
                .addComponent(newuser)
                .addComponent(Add_Question))
190             .addContainerGap(55, Short.MAX_VALUE))
191     );
192
193
194     jPanel5.setBorder(javax.swing.BorderFactory.
        createCompoundBorder(null, new javax.swing.border.
        SoftBevelBorder(javax.swing.border.BevelBorder.
        RAISED)));
195
196     jLabel4.setFont(new java.awt.Font("Liberation Serif",
        0, 15)); // NOI18N
197     jLabel4.setText("@mahabub");
198
199     javax.swing.GroupLayout jPanel5Layout = new javax.swing
        .GroupLayout(jPanel5);
200     jPanel5.setLayout(jPanel5Layout);
201     jPanel5Layout.setHorizontalGroup(
202         jPanel5Layout.createParallelGroup(javax.swing.
            GroupLayout.Alignment.LEADING)
203         .addGroup(jPanel5Layout.createSequentialGroup()
204             .addGroup(jPanel5Layout.createParallelGroup()
205                 .addGap(130, 130, 130)
206                 .addComponent(jLabel4)
207                 .addContainerGap(javax.swing.GroupLayout.
                    DEFAULT_SIZE, Short.MAX_VALUE))
208             .addContainerGap())
209     );
210     jPanel5Layout.setVerticalGroup(
211         jPanel5Layout.createParallelGroup(javax.swing.
            GroupLayout.Alignment.LEADING)
212         .addGroup(jPanel5Layout.createSequentialGroup()
213             .addGroup(jPanel5Layout.createParallelGroup.
                TRAILING, jPanel5Layout.createSequentialGroup()
                .addGap(0, 25, Short.MAX_VALUE)
                .addComponent(jLabel4))
214             .addContainerGap())
215     );
216
217     javax.swing.GroupLayout jPanel2Layout = new javax.swing
        .GroupLayout(jPanel2);
218     jPanel2.setLayout(jPanel2Layout);
219     jPanel2Layout.setHorizontalGroup(
        jPanel2Layout.createParallelGroup(javax.swing.
            GroupLayout.Alignment.LEADING)
        .addComponent(jPanel3, javax.swing.GroupLayout.

```

```

220         DEFAULT_SIZE, javax.swing.GroupLayout.
221         DEFAULT_SIZE, Short.MAX_VALUE)
222     .addGroup(jPanel2Layout.createSequentialGroup()
223         .addContainerGap()
224         .addComponent(jPanel4, javax.swing.GroupLayout.
225             DEFAULT_SIZE, javax.swing.GroupLayout.
226             DEFAULT_SIZE, Short.MAX_VALUE)
227         .addContainerGap())
228     .addComponent(jPanel5, javax.swing.GroupLayout.
229         DEFAULT_SIZE, javax.swing.GroupLayout.
230         DEFAULT_SIZE, Short.MAX_VALUE)
231 );
232 jPanel2Layout.setVerticalGroup(
233     jPanel2Layout.createParallelGroup(javax.swing.
234         GroupLayout.Alignment.LEADING)
235     .addGroup(jPanel2Layout.createSequentialGroup()
236         .addComponent(jPanel3, javax.swing.GroupLayout.
237             PREFERRED_SIZE, javax.swing.GroupLayout.
238             DEFAULT_SIZE, javax.swing.GroupLayout.
239             PREFERRED_SIZE)
240         .addPreferredGap(javax.swing.LayoutStyle.
241             ComponentPlacement.UNRELATED)
242         .addComponent(jPanel4, javax.swing.GroupLayout.
243             PREFERRED_SIZE, javax.swing.GroupLayout.
244             DEFAULT_SIZE, javax.swing.GroupLayout.
245             PREFERRED_SIZE)
246         .addPreferredGap(javax.swing.LayoutStyle.
247             ComponentPlacement.RELATED)
248         .addComponent(jPanel5, javax.swing.GroupLayout.
249             PREFERRED_SIZE, javax.swing.GroupLayout.
250             DEFAULT_SIZE, javax.swing.GroupLayout.
251             PREFERRED_SIZE)
252         .addGap(0, 0, Short.MAX_VALUE))
253 );
254 jPanel1.add(jPanel2, java.awt.BorderLayout.CENTER);
255
256 javax.swing.GroupLayout layout = new javax.swing.
257     GroupLayout(getContentPane());
258 getContentPane().setLayout(layout);
259 layout.setHorizontalGroup(
260     layout.createParallelGroup(javax.swing.GroupLayout.
261         Alignment.LEADING)
262     .addComponent(jPanel1, javax.swing.GroupLayout.
263         Alignment.TRAILING, javax.swing.GroupLayout.
264         DEFAULT_SIZE, javax.swing.GroupLayout.
265         DEFAULT_SIZE, Short.MAX_VALUE)
266 );
267 layout.setVerticalGroup(
268     layout.createParallelGroup(javax.swing.GroupLayout.
269         Alignment.LEADING)
270     .addComponent(jPanel1, javax.swing.GroupLayout.
271         DEFAULT_SIZE, javax.swing.GroupLayout.
272         DEFAULT_SIZE, Short.MAX_VALUE)
273 );
274
275 pack();
276 }// </editor-fold>

```

```

252
253 private void txtNameActionPerformed(java.awt.event.
      ActionEvent evt) {
254         userName = txtName.getText();
255         System.out.println("User entered: " + userName);
256     }
257
258 private void newUserActionPerformed(java.awt.event.
      ActionEvent evt) {
259         new Users().setVisible(true);
260         this.dispose();
261     }
262
263 private void LoginActionPerformed(java.awt.event.
      ActionEvent evt) {
264
265         userName = txtName.getText();
266         passWord = new String(txtpassword.getPassword());
267         try (Connection conn = connectToDatabase()) {
268             if (conn != null) {
269                 String sql = "SELECT * FROM users WHERE
      username = ? AND password = ?";
270                 try (PreparedStatement stmt = conn.
      prepareStatement(sql)) {
271                     stmt.setString(1, userName);
272                     stmt.setString(2, passWord);
273                     try (ResultSet rs = stmt.executeQuery()) {
274                         if (rs.next()) {
275                             new SubjectSelectionUI().setVisible
      (true);
276                         } else {
277                             JOptionPane.showMessageDialog(this,
      "Invalid username or password",
      "Error", JOptionPane.
      ERROR_MESSAGE);
278                         }
279                     }
280                 }
281             }
282         } catch (Exception e) {
283             JOptionPane.showMessageDialog(this, "Error: " + e.
      getMessage(), "Error", JOptionPane.ERROR_MESSAGE
      );
284         }
285     }
286
287 private void txtpasswordActionPerformed(java.awt.event.
      ActionEvent evt) {
288
289     }
290
291 private void Add_QuestionActionPerformed(java.awt.event.
      ActionEvent evt) {
292         new Questions_add();
293         this.dispose();
294     }
295
296 public static void main(String args[]) {

```

```

297     try {
298         for (javax.swing.UIManager.LookAndFeelInfo info :
                javax.swing.UIManager.getInstalledLookAndFeels()) {
299             if ("Nimbus".equals(info.getName())) {
300                 javax.swing.UIManager.setLookAndFeel(info.
                    getClassName());
301                 break;
302             }
303         }
304     } catch (ClassNotFoundException ex) {
305         java.util.logging.Logger.getLogger(login.class.
            getName()).log(java.util.logging.Level.SEVERE,
                null, ex);
306     } catch (InstantiationException ex) {
307         java.util.logging.Logger.getLogger(login.class.
            getName()).log(java.util.logging.Level.SEVERE,
                null, ex);
308     } catch (IllegalAccessException ex) {
309         java.util.logging.Logger.getLogger(login.class.
            getName()).log(java.util.logging.Level.SEVERE,
                null, ex);
310     } catch (javax.swing.UnsupportedLookAndFeelException ex
        ) {
311         java.util.logging.Logger.getLogger(login.class.
            getName()).log(java.util.logging.Level.SEVERE,
                null, ex);
312     }
313     java.awt.EventQueue.invokeLater(new Runnable() {
314         public void run() {
315             new login().setVisible(true);
316         }
317     });
318 }
319
320 // Variables declaration - do not modify
321 private javax.swing.JButton Add_Question;
322 private javax.swing.JButton Login;
323 private javax.swing.JLabel Password;
324 private javax.swing.JLabel jLabel3;
325 private javax.swing.JLabel jLabel4;
326 private javax.swing.JPanel jPanel1;
327 private javax.swing.JPanel jPanel2;
328 private javax.swing.JPanel jPanel3;
329 private javax.swing.JPanel jPanel4;
330 private javax.swing.JPanel jPanel5;
331 private javax.swing.JPanel jPanel6;
332 private javax.swing.JPanel jPanel7;
333 private javax.swing.JButton newuser;
334 private javax.swing.JTextField txtName;
335 private javax.swing.JPasswordField txtpassword;
336 private javax.swing.JLabel user;
337 // End of variables declaration
338 }

```

Listing 2.3: login

```

1 package quizzes;

```

```

2 import java.security.MessageDigest;
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.PreparedStatement;
6 import java.sql.ResultSet;
7 import java.sql.SQLException;
8 import javax.swing.JOptionPane;
9 import javax.swing.table.DefaultTableModel;
10
11 public class Users extends javax.swing.JFrame {
12     private String userName;
13     private String passWord;
14     private Connection connectToDatabase() {
15         Connection conn = null;
16         try {
17             String url = "jdbc:mysql://127.0.0.1:3306/last?
                zeroDateTimeBehavior=CONVERT_TO_NULL"; // Your
                DB URL
18             String dbUser = "root";
19             String dbPassword = "";
20             conn = DriverManager.getConnection(url, dbUser,
                dbPassword);
21         } catch (Exception e) {
22             JOptionPane.showMessageDialog(this, "Database
                connection failed: " + e.getMessage(), "Error",
                JOptionPane.ERROR_MESSAGE);
23         }
24         return conn;
25     }
26     private void insertUser(String userName, String password) {
27         try (Connection conn = connectToDatabase()) {
28             if (conn != null) {
29                 String sql = "INSERT INTO users (username, password
                    ) VALUES (?, ?)";
30                 PreparedStatement stmt = conn.prepareStatement(sql)
                    ;
31                 stmt.setString(1, userName);
32                 stmt.setString(2, password);
33
34                 int rowsInserted = stmt.executeUpdate();
35                 if (rowsInserted > 0) {
36                     JOptionPane.showMessageDialog(this, "User
                        registered successfully!", "Success",
                        JOptionPane.INFORMATION_MESSAGE);
37                 } else {
38                     JOptionPane.showMessageDialog(this, "Failed to
                        register user", "Error", JOptionPane.
                        ERROR_MESSAGE);
39                 }
40             }
41         } catch (Exception e) {
42             JOptionPane.showMessageDialog(this, "Error: " + e.
                getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
43         }
44     }
45
46     public Users() {
47         initComponents();

```

```

48     }
49     @SuppressWarnings("unchecked")
50     // <editor-fold defaultstate="collapsed" desc="Generated
    Code">
51     private void initComponents() {
52
53         jPanel6 = new javax.swing.JPanel();
54         jLabel3 = new javax.swing.JLabel();
55         jTextField3 = new javax.swing.JTextField();
56         jScrollPane1 = new javax.swing.JScrollPane();
57         jTable1 = new javax.swing.JTable();
58         jPanel1 = new javax.swing.JPanel();
59         jPanel2 = new javax.swing.JPanel();
60         jPanel3 = new javax.swing.JPanel();
61         jPanel7 = new javax.swing.JPanel();
62         txtpassword = new javax.swing.JLabel();
63         password = new javax.swing.JTextField();
64         save_value = new javax.swing.JButton();
65         Go_to_login = new javax.swing.JButton();
66         username = new javax.swing.JTextField();
67         txtName = new javax.swing.JLabel();
68         Edit = new javax.swing.JButton();
69         Delete = new javax.swing.JButton();
70         jPanel9 = new javax.swing.JPanel();
71         jScrollPane2 = new javax.swing.JScrollPane();
72         UserTable = new javax.swing.JTable();
73         jPanel10 = new javax.swing.JPanel();
74
75         jLabel3.setText("Users ID :");
76
77         jTextField3.setBorder(javax.swing.BorderFactory.
            createEtchedBorder());
78         jTextField3.addActionListener(new java.awt.event.
            ActionListener() {
79             public void actionPerformed(java.awt.event.
               (ActionEvent evt) {
80                 jTextField3ActionPerformed(evt);
81             }
82         });
83
84         javax.swing.GroupLayout jPanel6Layout = new javax.swing
            .GroupLayout(jPanel6);
85         jPanel6.setLayout(jPanel6Layout);
86         jPanel6Layout.setHorizontalGroup(
87             jPanel6Layout.createParallelGroup(javax.swing.
                GroupLayout.Alignment.LEADING)
88                 .addGroup(jPanel6Layout.createSequentialGroup()
89                     .addGroup(jPanel6Layout.createParallelGroup()
90                         .addContainerGap()
91                         .addComponent(jLabel3)
92                         .addGap(40, 40, 40)
93                         .addComponent(jTextField3, javax.swing.
                            GroupLayout.PREFERRED_SIZE, javax.swing.
                                GroupLayout.DEFAULT_SIZE, javax.swing.
                                    GroupLayout.PREFERRED_SIZE)
94                         .addContainerGap())
95                     .addContainerGap())
96                 .addGroup(jPanel6Layout.createSequentialGroup()

```

```

100         GroupLayout.Alignment.LEADING)
101     .addGroup(jPanel6Layout.createSequentialGroup()
102         .addContainerGap()
103         .addGroup(jPanel6Layout.createParallelGroup(
104             javax.swing.GroupLayout.Alignment.BASELINE)
105             .addComponent(jLabel3)
106             .addComponent(jTextField3, javax.swing.
107                 GroupLayout.PREFERRED_SIZE, javax.swing.
108                 GroupLayout.DEFAULT_SIZE, javax.swing.
109                 GroupLayout.PREFERRED_SIZE))
110         .addContainerGap())
111 );
112
113 jTable1.setModel(new javax.swing.table.
114     DefaultTableModel(
115         new Object [][] {
116             {null, null, null, null},
117             {null, null, null, null},
118             {null, null, null, null},
119             {null, null, null, null}
120         },
121         new String [] {
122             "Title 1", "Title 2", "Title 3", "Title 4"
123         }
124     ));
125 jScrollPane1.setViewportView(jTable1);
126
127 setDefaultCloseOperation(javax.swing.WindowConstants.
128     EXIT_ON_CLOSE);
129
130 jPanel1.setLayout(new java.awt.BorderLayout());
131
132 jPanel2.setBorder(javax.swing.BorderFactory.
133     createBevelBorder(javax.swing.border.BevelBorder.
134         RAISED));
135
136 javax.swing.GroupLayout jPanel2Layout = new javax.swing.
137     GroupLayout(jPanel2);
138 jPanel2.setLayout(jPanel2Layout);
139 jPanel2Layout.setHorizontalGroup(
140     jPanel2Layout.createParallelGroup(javax.swing.
141         GroupLayout.Alignment.LEADING)
142         .addGap(0, 0, Short.MAX_VALUE)
143     );
144 jPanel2Layout.setVerticalGroup(
145     jPanel2Layout.createParallelGroup(javax.swing.
146         GroupLayout.Alignment.LEADING)
147         .addGap(0, 29, Short.MAX_VALUE)
148     );
149
150 jPanel3.setBorder(javax.swing.BorderFactory.
151     createEtchedBorder());
152
153 txtpassword.setText("Password :");
154
155 password.setBorder(javax.swing.BorderFactory.
156     createEtchedBorder());
157 password.addActionListener(new java.awt.event.

```

```

141     ActionListener() {
142         public void actionPerformed(java.awt.event.
143             ActionEvent evt) {
144             passwordActionPerformed(evt);
145         }
146     });
147
148     javax.swing.GroupLayout jPanel7Layout = new javax.swing
149         .GroupLayout(jPanel7);
150     jPanel7.setLayout(jPanel7Layout);
151     jPanel7Layout.setHorizontalGroup(
152         jPanel7Layout.createParallelGroup(javax.swing.
153             GroupLayout.Alignment.LEADING)
154             .addGroup(jPanel7Layout.createSequentialGroup()
155                 .addComponent(txtpassword)
156                 .addPreferredGap(javax.swing.LayoutStyle.
157                     ComponentPlacement.RELATED)
158                 .addComponent(password, javax.swing.GroupLayout.
159                     PREFERRED_SIZE, 210, javax.swing.
160                     GroupLayout.PREFERRED_SIZE)
161                 .addContainerGap(javax.swing.GroupLayout.PREFERRED_SIZE))
162             .addGroup(jPanel7Layout.createParallelGroup(javax.swing.
163                 GroupLayout.Alignment.LEADING)
164                 .addGroup(jPanel7Layout.createSequentialGroup()
165                     .addComponent(txtpassword)
166                     .addPreferredGap(javax.swing.LayoutStyle.
167                         ComponentPlacement.RELATED)
168                     .addComponent(password, javax.swing.
169                         GroupLayout.PREFERRED_SIZE, javax.swing.
170                         GroupLayout.DEFAULT_SIZE, javax.swing.
171                         GroupLayout.PREFERRED_SIZE))
172                 .addGroup(jPanel7Layout.createSequentialGroup()
173                     .addComponent(password, javax.swing.
174                         GroupLayout.PREFERRED_SIZE, javax.swing.
175                         GroupLayout.DEFAULT_SIZE, javax.swing.
176                         GroupLayout.PREFERRED_SIZE))
177                 .addGroup(jPanel7Layout.createSequentialGroup()
178                     .addComponent(txtpassword)
179                     .addPreferredGap(javax.swing.LayoutStyle.
180                         ComponentPlacement.RELATED)
181                     .addComponent(password, javax.swing.
182                         GroupLayout.PREFERRED_SIZE, javax.swing.
183                         GroupLayout.DEFAULT_SIZE, javax.swing.
184                         GroupLayout.PREFERRED_SIZE))
185             );
186
187     save_value.setText("Save");
188     save_value.addActionListener(new java.awt.event.
189         ActionListener() {
190             public void actionPerformed(java.awt.event.
191                 ActionEvent evt) {
192                 save_valueActionPerformed(evt);
193             }
194         });
195
196     Go_to_login.setText("Go to Login");
197     Go_to_login.addActionListener(new java.awt.event.
198         ActionListener() {
199             public void actionPerformed(java.awt.event.
200                 ActionEvent evt) {
201                 Go_to_loginActionPerformed(evt);
202             }
203         });
204
205     username.setBorder(javax.swing.BorderFactory.
206         createEtchedBorder());
207     username.addActionListener(new java.awt.event.

```

```

181         ActionListener() {
182             public void actionPerformed(java.awt.event.
183                 ActionEvent evt) {
184                 usernameActionPerformed(evt);
185             }
186         });
187
188         txtName.setText("Name :");
189
190         Edit.setText("Edit");
191         Edit.addActionListener(new java.awt.event.
192             ActionListener() {
193                 public void actionPerformed(java.awt.event.
194                     ActionEvent evt) {
195                     EditActionPerformed(evt);
196                 }
197             });
198
199         Delete.setText("Delete");
200         Delete.addActionListener(new java.awt.event.
201             ActionListener() {
202                 public void actionPerformed(java.awt.event.
203                     ActionEvent evt) {
204                     DeleteActionPerformed(evt);
205                 }
206             });
207
208         javax.swing.GroupLayout jPanel3Layout = new javax.swing
209             .GroupLayout(jPanel3);
210         jPanel3.setLayout(jPanel3Layout);
211         jPanel3Layout.setHorizontalGroup(
212             jPanel3Layout.createParallelGroup(javax.swing.
213                 GroupLayout.Alignment.LEADING)
214                 .addGroup(jPanel3Layout.createSequentialGroup()
215                     .add(jPanel3Layout.createParallelGroup(
216                         javax.swing.GroupLayout.Alignment.LEADING)
217                         .addGroup(jPanel3Layout.createSequentialGroup()
218                             .add(txtName, javax.swing.
219                                 GroupLayout.PREFERRED_SIZE, 52,
220                                 javax.swing.GroupLayout.
221                                     PREFERRED_SIZE)
222                             .addPreferredGap(javax.swing.
223                                 LayoutStyle.ComponentPlacement.
224                                     UNRELATED)
225                             .add(username, javax.swing.
226                                 GroupLayout.PREFERRED_SIZE, 211,
227                                 javax.swing.GroupLayout.
228                                     PREFERRED_SIZE)))
229                         .addContainerGap(203, Short.MAX_VALUE))
230                     .addGroup(javax.swing.GroupLayout.Alignment.
231                         TRAILING, jPanel3Layout.createSequentialGroup()
232                             .addGap(15, 15, 15)

```

```

217         .addComponent(save_value)
218         .addGap(35, 35, 35)
219         .addComponent(Edit)
220         .addPreferredGap(javax.swing.LayoutStyle.
                ComponentPlacement.RELATED, javax.swing.
                GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
221         .addComponent(Delete)
222         .addGap(53, 53, 53)
223         .addComponent(Go_to_login)
224         .addGap(27, 27, 27))
225     );
226     JPanel3Layout.setVerticalGroup(
227         JPanel3Layout.createParallelGroup(javax.swing.
                GroupLayout.Alignment.LEADING)
228         .addGroup(JPanel3Layout.createSequentialGroup())
229         .addGap(18, 18, 18)
230         .addGroup(JPanel3Layout.createParallelGroup(
                javax.swing.GroupLayout.Alignment.BASELINE)
                .addComponent(txtName)
231                .addComponent(username, javax.swing.
                GroupLayout.PREFERRED_SIZE, javax.swing.
                GroupLayout.DEFAULT_SIZE, javax.swing.
                GroupLayout.PREFERRED_SIZE))
232                .addPreferredGap(javax.swing.LayoutStyle.
                ComponentPlacement.RELATED)
233                .addComponent(JPanel7, javax.swing.GroupLayout.
                PREFERRED_SIZE, javax.swing.GroupLayout.
                DEFAULT_SIZE, javax.swing.GroupLayout.
                PREFERRED_SIZE)
234                .addGap(18, 18, 18)
235                .addGroup(JPanel3Layout.createParallelGroup(
                javax.swing.GroupLayout.Alignment.LEADING)
                .addGroup(JPanel3Layout.createParallelGroup
                (javax.swing.GroupLayout.Alignment.
                BASELINE)
                .addComponent(Go_to_login)
236                .addComponent(Delete))
237                .addGroup(JPanel3Layout.createParallelGroup
                (javax.swing.GroupLayout.Alignment.
                BASELINE)
                .addComponent(save_value)
238                .addComponent(Edit)))
239                .addContainerGap(16, Short.MAX_VALUE))
240    );
241
242     JPanel9.setBorder(javax.swing.BorderFactory.
243         createEtchedBorder());
244
245     UserTable.setModel(new javax.swing.table.
246         DefaultTableModel(
247             new Object [][] {
248                 },
249             new String [] {
250                 "Name", "Password"
251             }
252         ));
253     UserTable.addAncestorListener(new javax.swing.event.

```

```

257     AncestorListener() {
258         public void ancestorAdded(javax.swing.event.
259             AncestorEvent evt) {
260             UserTableAncestorAdded(evt);
261         }
262         public void ancestorMoved(javax.swing.event.
263             AncestorEvent evt) {
264         }
265         public void ancestorRemoved(javax.swing.event.
266             AncestorEvent evt) {
267         }
268     });
269     JScrollPane2.setViewportViewView(UserTable);
270
271     javax.swing.GroupLayout jPanel9Layout = new javax.swing
272         .GroupLayout(jPanel9);
273     jPanel9.setLayout(jPanel9Layout);
274     jPanel9Layout.setHorizontalGroup(
275         jPanel9Layout.createParallelGroup(javax.swing.
276             GroupLayout.Alignment.LEADING)
277         .addComponent(jScrollPane2, javax.swing.GroupLayout.
278             Alignment.TRAILING, javax.swing.GroupLayout.
279             DEFAULT_SIZE, 488, Short.MAX_VALUE)
280     );
281     jPanel9Layout.setVerticalGroup(
282         jPanel9Layout.createParallelGroup(javax.swing.
283             GroupLayout.Alignment.LEADING)
284         .addComponent(jScrollPane2, javax.swing.GroupLayout.
285             DEFAULT_SIZE, 162, Short.MAX_VALUE)
286     );
287
288     jPanel10.setBorder(javax.swing.BorderFactory.
289         createBevelBorder(javax.swing.border.BevelBorder.
290             RAISED));
291
292     javax.swing.GroupLayout jPanel10Layout = new javax.
293         swing.GroupLayout(jPanel10);
294     jPanel10.setLayout(jPanel10Layout);
295     jPanel10Layout.setHorizontalGroup(
296         jPanel10Layout.createParallelGroup(javax.swing.
297             GroupLayout.Alignment.LEADING)
298         .addGap(0, 0, Short.MAX_VALUE)
299     );
300     jPanel10Layout.setVerticalGroup(
301         jPanel10Layout.createParallelGroup(javax.swing.
302             GroupLayout.Alignment.LEADING)
303         .addGap(0, 28, Short.MAX_VALUE)
304     );
305
306     javax.swing.GroupLayout layout = new javax.swing.
307         GroupLayout(getContentPane());
308     getContentPane().setLayout(layout);
309     layout.setHorizontalGroup(
310         layout.createParallelGroup(javax.swing.GroupLayout.
311             Alignment.LEADING)
312         .addComponent(jPanel1, javax.swing.GroupLayout.
313             DEFAULT_SIZE, javax.swing.GroupLayout.
314             DEFAULT_SIZE, Short.MAX_VALUE)

```

```

296 .addComponent(jPanel2, javax.swing.GroupLayout.
    DEFAULT_SIZE, javax.swing.GroupLayout.
    DEFAULT_SIZE, Short.MAX_VALUE)
297 .addComponent(jPanel3, javax.swing.GroupLayout.
    DEFAULT_SIZE, javax.swing.GroupLayout.
    DEFAULT_SIZE, Short.MAX_VALUE)
298 .addGroup(layout.createSequentialGroup()
299     .addComponent(jPanel9, javax.swing.GroupLayout.
    DEFAULT_SIZE, javax.swing.GroupLayout.
    DEFAULT_SIZE, Short.MAX_VALUE)
300     .addContainerGap())
301 .addComponent(jPanel10, javax.swing.GroupLayout.
    Alignment.TRAILING, javax.swing.GroupLayout.
    DEFAULT_SIZE, javax.swing.GroupLayout.
    DEFAULT_SIZE, Short.MAX_VALUE)
302 );
303 layout.setVerticalGroup(
304     layout.createParallelGroup(javax.swing.GroupLayout.
    Alignment.LEADING)
305 .addGroup(javax.swing.GroupLayout.Alignment.
    TRAILING, layout.createSequentialGroup()
306     .addComponent(jPanel2, javax.swing.GroupLayout.
    PREFERRED_SIZE, javax.swing.GroupLayout.
    DEFAULT_SIZE, javax.swing.GroupLayout.
    PREFERRED_SIZE)
307     .addPreferredGap(javax.swing.LayoutStyle.
    ComponentPlacement.RELATED)
308     .addComponent(jPanel3, javax.swing.GroupLayout.
    PREFERRED_SIZE, javax.swing.GroupLayout.
    DEFAULT_SIZE, javax.swing.GroupLayout.
    PREFERRED_SIZE)
309     .addPreferredGap(javax.swing.LayoutStyle.
    ComponentPlacement.UNRELATED)
310     .addComponent(jPanel9, javax.swing.GroupLayout.
    PREFERRED_SIZE, javax.swing.GroupLayout.
    DEFAULT_SIZE, javax.swing.GroupLayout.
    PREFERRED_SIZE)
311     .addPreferredGap(javax.swing.LayoutStyle.
    ComponentPlacement.RELATED, javax.swing.
    GroupLayout.DEFAULT_SIZE, Short.MAX_VALUE)
312     .addComponent(jPanel11, javax.swing.GroupLayout.
    PREFERRED_SIZE, javax.swing.GroupLayout.
    DEFAULT_SIZE, javax.swing.GroupLayout.
    PREFERRED_SIZE)
313     .addPreferredGap(javax.swing.LayoutStyle.
    ComponentPlacement.RELATED)
314     .addComponent(jPanel10, javax.swing.GroupLayout.
    PREFERRED_SIZE, javax.swing.GroupLayout.
    DEFAULT_SIZE, javax.swing.GroupLayout.
    PREFERRED_SIZE))
315 );
316
317     pack();
318 }// </editor-fold>
319
320 private void jTextField3ActionPerformed(java.awt.event.
    ActionEvent evt) {
321

```

```

322     }
323
324     private void usernameActionPerformed(java.awt.event.
        ActionEvent evt) {
325
326
327     }
328
329     private void save_valueActionPerformed(java.awt.event.
        ActionEvent evt) {
330         userName = username.getText();
331         passWord = new String(password.getText());
332         if (userName.isEmpty() || passWord.isEmpty()) {
333             JOptionPane.showMessageDialog(this, "Please fill in
                both username and password", "Error", JOptionPane.
                ERROR_MESSAGE);
334             return;
335         }
336         try (Connection conn = connectToDatabase()) {
337             if (conn != null) {
338                 String sql = "INSERT INTO users (username, password
                    ) VALUES (?, ?)";
339                 PreparedStatement stmt = conn.prepareStatement(sql)
                    ;
340                 stmt.setString(1, userName);
341                 stmt.setString(2, passWord);
342
343                 int rowsInserted = stmt.executeUpdate();
344                 if (rowsInserted > 0) {
345                     JOptionPane.showMessageDialog(this, "User
                        registered successfully!", "Success",
                        JOptionPane.INFORMATION_MESSAGE);
346
347                     refreshUserTable();
348                     new login().setVisible(true);
349                     this.dispose();
350                 } else {
351                     JOptionPane.showMessageDialog(this, "User
                        registration failed", "Error", JOptionPane.
                        ERROR_MESSAGE);
352                 }
353             }
354         } catch (Exception e) {
355             JOptionPane.showMessageDialog(this, "Error: " + e.
                getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
356         }
357     }
358
359     private void refreshUserTable() {
360         DefaultTableModel model = (DefaultTableModel) UserTable.
            getModel();
361         model.setRowCount(0);
362
363         try (Connection conn = connectToDatabase()) {
364             if (conn != null) {
365                 String sql = "SELECT username, password FROM users"
                    ;
366                 PreparedStatement stmt = conn.prepareStatement(sql)

```

```

367         ;
368         ResultSet rs = stmt.executeQuery();
369         while (rs.next()) {
370             String username = rs.getString("username");
371             String password = rs.getString("password");
372             model.addRow(new Object[] { username, password
373             });
374         }
375     } catch (SQLException e) {
376         JOptionPane.showMessageDialog(this, "Error: " + e.
377             getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
378     }
379 }
380
381 private void Go_to_loginActionPerformed(java.awt.event.
382     ActionEvent evt) {
383     new login().setVisible(true);
384     this.dispose();
385 }
386
387 private void passwordActionPerformed(java.awt.event.
388     ActionEvent evt) {
389 }
390
391 private void UserTableAncestorAdded(javax.swing.event.
392     AncestorEvent evt) {
393     DefaultTableModel model = (DefaultTableModel) UserTable.
394         getModel();
395     model.setRowCount(0);
396     try (Connection conn = connectToDatabase()) {
397         if (conn != null) {
398             String sql = "SELECT username, password FROM users"
399             ;
400             PreparedStatement stmt = conn.prepareStatement(sql)
401             ;
402             ResultSet rs = stmt.executeQuery();
403
404             while (rs.next()) {
405                 String username = rs.getString("username");
406                 String password = rs.getString("password");
407                 model.addRow(new Object[] { username, password
408                 });
409             }
410         }
411     } catch (SQLException e) {
412         JOptionPane.showMessageDialog(this, "Error: " + e.
413             getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
414     }
415 }
416
417 private void EditActionPerformed(java.awt.event.ActionEvent
418     evt) {
419     int selectedRow = UserTable.getSelectedRow();
420
421     if (selectedRow == -1) {
422         JOptionPane.showMessageDialog(this, "Please select

```

```

        a user to edit.", "Error", JOptionPane.
        ERROR_MESSAGE);
413     return;
414 }
415 String oldUserName = (String) UserTable.getValueAt(
        selectedRow, 0);
416 String oldPassword = (String) UserTable.getValueAt(
        selectedRow, 1);
417
418 String newUserName = username.getText();
419 String newPassword = password.getText();
420
421 if (newUserName.isEmpty() || newPassword.isEmpty()) {
422     JOptionPane.showMessageDialog(this, "Please enter both
        username and password.", "Error", JOptionPane.
        ERROR_MESSAGE);
423     return;
424 }
425
426 int confirmation = JOptionPane.showConfirmDialog(this, "Are
        you sure you want to update the user: " + oldUserName +
        "?", "Confirm Edit", JOptionPane.YES_NO_OPTION);
427
428 if (confirmation == JOptionPane.YES_OPTION) {
429     try (Connection conn = connectToDatabase()) {
430         if (conn != null) {
431             String sql = "UPDATE users SET username = ?,
                password = ? WHERE username = ?";
432             PreparedStatement stmt = conn.prepareStatement(
                sql);
433             stmt.setString(1, newUserName);
434             stmt.setString(2, newPassword);
435             stmt.setString(3, oldUserName);
436
437             int rowsAffected = stmt.executeUpdate();
438             if (rowsAffected > 0) {
439                 DefaultTableModel model = (
                    DefaultTableModel) UserTable.getModel();
440                 model.setValueAt(newUserName, selectedRow,
                    0);
441                 model.setValueAt(newPassword, selectedRow,
                    1);
442
443                 JOptionPane.showMessageDialog(this, "User
                    updated successfully!", "Success",
                    JOptionPane.INFORMATION_MESSAGE);
444             } else {
445                 JOptionPane.showMessageDialog(this, "Error
                    updating user.", "Error", JOptionPane.
                    ERROR_MESSAGE);
446             }
447         }
448     } catch (SQLException e) {
449         JOptionPane.showMessageDialog(this, "Error updating
            user: " + e.getMessage(), "Error", JOptionPane.
            ERROR_MESSAGE);
450     }
451 }

```

```

452     }
453
454     private void DeleteActionPerformed(java.awt.event.
        ActionEvent evt) {
455
456         int selectedRow = UserTable.getSelectedRow();
457
458         if (selectedRow == -1) {
459             JOptionPane.showMessageDialog(this, "Please select a
                user to delete.", "Error", JOptionPane.ERROR_MESSAGE
            );
460             return;
461         }
462         String userNameToDelete = (String) UserTable.getValueAt(
            selectedRow, 0);
463
464         int confirmation = JOptionPane.showConfirmDialog(this, "Are
            you sure you want to delete the user: " +
            userNameToDelete + "?", "Confirm Deletion", JOptionPane.
            YES_NO_OPTION);
465
466         if (confirmation == JOptionPane.YES_OPTION) {
467             try (Connection conn = connectToDatabase()) {
468                 if (conn != null) {
469                     String sql = "DELETE FROM users WHERE username
                        = ?";
470                     PreparedStatement stmt = conn.prepareStatement(
                        sql);
471                     stmt.setString(1, userNameToDelete);
472
473                     int rowsAffected = stmt.executeUpdate();
474                     if (rowsAffected > 0) {
475                         DefaultTableModel model = (
                            DefaultTableModel) UserTable.getModel();
476                         model.removeRow(selectedRow);
477
478                         JOptionPane.showMessageDialog(this, "User
                            deleted successfully!", "Success",
                            JOptionPane.INFORMATION_MESSAGE);
479                     } else {
480                         JOptionPane.showMessageDialog(this, "User
                            not found.", "Error", JOptionPane.
                            ERROR_MESSAGE);
481                     }
482                 }
483             } catch (SQLException e) {
484                 JOptionPane.showMessageDialog(this, "Error deleting
                    user: " + e.getMessage(), "Error", JOptionPane.
                    ERROR_MESSAGE);
485             }
486         }
487     }
488
489     public static void main(String args[]) {
490         try {
491             for (javax.swing.UIManager.LookAndFeelInfo info :
                javax.swing.UIManager.getInstalledLookAndFeels()
            ) {

```

```

492         if ("Nimbus".equals(info.getName())) {
493             javax.swing.UIManager.setLookAndFeel(info.
494                 getClassName());
495             break;
496         }
497     } catch (ClassNotFoundException ex) {
498         java.util.logging.Logger.getLogger(Users.class.
499             getName()).log(java.util.logging.Level.SEVERE,
500                 null, ex);
501     } catch (InstantiationException ex) {
502         java.util.logging.Logger.getLogger(Users.class.
503             getName()).log(java.util.logging.Level.SEVERE,
504                 null, ex);
505     } catch (IllegalAccessException ex) {
506         java.util.logging.Logger.getLogger(Users.class.
507             getName()).log(java.util.logging.Level.SEVERE,
508                 null, ex);
509     } catch (javax.swing.UnsupportedLookAndFeelException ex
510     ) {
511         java.util.logging.Logger.getLogger(Users.class.
512             getName()).log(java.util.logging.Level.SEVERE,
513                 null, ex);
514     }
515
516     java.awt.EventQueue.invokeLater(new Runnable() {
517         public void run() {
518             new Users().setVisible(true);
519         }
520     });
521 }
522
523 // Variables declaration - do not modify
524 private javax.swing.JButton Delete;
525 private javax.swing.JButton Edit;
526 private javax.swing.JButton Go_to_login;
527 private javax.swing.JTable UserTable;
528 private javax.swing.JLabel jLabel3;
529 private javax.swing.JPanel jPanel1;
530 private javax.swing.JPanel jPanel10;
531 private javax.swing.JPanel jPanel2;
532 private javax.swing.JPanel jPanel3;
533 private javax.swing.JPanel jPanel6;
534 private javax.swing.JPanel jPanel7;
535 private javax.swing.JPanel jPanel9;
536 private javax.swing.JScrollPane jScrollPane1;
537 private javax.swing.JScrollPane jScrollPane2;
538 private javax.swing.JTable jTable1;
539 private javax.swing.JTextField jTextField3;
540 private javax.swing.JTextField password;
541 private javax.swing.JButton save_value;
542 private javax.swing.JLabel txtName;
543 private javax.swing.JLabel txtpassword;
544 private javax.swing.JTextField username;
545 // End of variables declaration
546 }

```

Listing 2.4: User

```

1      package quizzes;
2
3      import java.sql.*;
4      import java.awt.*;
5      import java.awt.event.*;
6      import javax.swing.*;
7      import java.util.ArrayList;
8      import java.util.List;
9      import javax.swing.table.DefaultTableModel;
10
11     public class Questions_add extends JFrame implements
        ActionListener {
12         private Connection conn;
13         private PreparedStatement pstmt;
14         private ResultSet rs;
15
16         private List<Question> questionsList;
17         private JTable questionTable;
18         private DefaultTableModel tableModel;
19
20         private JButton addButton, saveButton, editButton,
            deleteButton;
21
22         public Questions_add() {
23             connectToDatabase();
24             loadQuestionsFromDatabase();
25             openAdminFrame();
26         }
27
28         public void connectToDatabase() {
29             try {
30                 Class.forName("com.mysql.cj.jdbc.Driver");
31                 conn = DriverManager.getConnection("jdbc:mysql://
                    localhost:3306/last?zeroDateTimeBehavior=
                    CONVERT_TO_NULL", "root", "");
32             } catch (Exception e) {
33                 JOptionPane.showMessageDialog(this, "Database
                    connection failed: " + e.getMessage());
34             }
35         }
36
37         public void loadQuestionsFromDatabase() {
38             questionsList = new ArrayList<>();
39             try {
40                 String query = "SELECT * FROM question";
41                 pstmt = conn.prepareStatement(query);
42                 rs = pstmt.executeQuery();
43                 while (rs.next()) {
44                     String questionText = rs.getString("question");
45                     String option1Text = rs.getString("option_1");
46                     String option2Text = rs.getString("option_2");
47                     String option3Text = rs.getString("option_3");
48                     String option4Text = rs.getString("option_4");
49                     int correctAnswer = rs.getInt("correct_answer")
                        ;
50
51                     questionsList.add(new Question(questionText,

```

```

        option1Text, option2Text, option3Text,
        option4Text, correctAnswer));
52     }
53 } catch (SQLException e) {
54     JOptionPane.showMessageDialog(this, "Error loading
55         questions from database: " + e.getMessage());
56 }
57
58 public void openAdminFrame() {
59     setTitle("Question Management System");
60     setBounds(300, 90, 900, 600);
61     setDefaultCloseOperation(EXIT_ON_CLOSE);
62     setResizable(false);
63
64     Container container = getContentPane();
65     container.setLayout(null);
66
67     JLabel title = new JLabel("Question Management");
68     title.setFont(new Font("Arial", Font.PLAIN, 24));
69     title.setSize(300, 30);
70     title.setLocation(300, 30);
71     container.add(title);
72
73     // Table for displaying questions
74     tableModel = new DefaultTableModel(new String[]{"
75         Question", "Option 1", "Option 2", "Option 3", "
76         Option 4", "Correct Answer"}, 0);
77     questionTable = new JTable(tableModel);
78     JScrollPane scrollPane = new JScrollPane(questionTable)
79     ;
80     scrollPane.setBounds(50, 100, 800, 300);
81     container.add(scrollPane);
82
83     addButton = new JButton("Add Question");
84     addButton.setFont(new Font("Arial", Font.PLAIN, 15));
85     addButton.setSize(150, 30);
86     addButton.setLocation(50, 450);
87     addButton.addActionListener(this);
88     container.add(addButton);
89
90     editButton = new JButton("Edit Question");
91     editButton.setFont(new Font("Arial", Font.PLAIN, 15));
92     editButton.setSize(150, 30);
93     editButton.setLocation(220, 450);
94     editButton.addActionListener(this);
95     container.add(editButton);
96
97     deleteButton = new JButton("Delete Question");
98     deleteButton.setFont(new Font("Arial", Font.PLAIN, 15))
99     ;
100     deleteButton.setSize(150, 30);
101     deleteButton.setLocation(390, 450);
102     deleteButton.addActionListener(this);
103     container.add(deleteButton);
104
105     saveButton = new JButton("Save Changes");
106     saveButton.setFont(new Font("Arial", Font.PLAIN, 15));

```

```

103         saveButton.setSize(150, 30);
104         saveButton.setLocation(560, 450);
105         saveButton.addActionListener(this);
106         container.add(saveButton);
107
108         loadQuestionsIntoTable();
109         setVisible(true);
110     }
111
112     public void loadQuestionsIntoTable() {
113         tableModel.setRowCount(0); // Clear table
114         for (Question question : questionsList) {
115             tableModel.addRow(new Object[]{
116                 question.getQuestionText(),
117                 question.getOption1(),
118                 question.getOption2(),
119                 question.getOption3(),
120                 question.getOption4(),
121                 question.getCorrectAnswer()
122             });
123         }
124     }
125
126     @Override
127     public void actionPerformed(ActionEvent e) {
128         if (e.getSource() == addButton) {
129             addQuestion();
130         } else if (e.getSource() == editButton) {
131             editQuestion();
132         } else if (e.getSource() == deleteButton) {
133             deleteQuestion();
134         } else if (e.getSource() == saveButton) {
135             saveChanges();
136         }
137     }
138
139     private void addQuestion() {
140         JTextField questionField = new JTextField();
141         JTextField option1Field = new JTextField();
142         JTextField option2Field = new JTextField();
143         JTextField option3Field = new JTextField();
144         JTextField option4Field = new JTextField();
145         JTextField correctAnswerField = new JTextField();
146
147         Object[] fields = {
148             "Question:", questionField,
149             "Option 1:", option1Field,
150             "Option 2:", option2Field,
151             "Option 3:", option3Field,
152             "Option 4:", option4Field,
153             "Correct Answer (1-4):", correctAnswerField
154         };
155
156         int option = JOptionPane.showConfirmDialog(this, fields,
157             "Add New Question", JOptionPane.OK_CANCEL_OPTION);
158         if (option == JOptionPane.OK_OPTION) {
159             try {
160                 String questionText = questionField.getText();

```

```

160         String option1Text = option1Field.getText();
161         String option2Text = option2Field.getText();
162         String option3Text = option3Field.getText();
163         String option4Text = option4Field.getText();
164         int correctAnswer = Integer.parseInt(
165             correctAnswerField.getText());
166
167         String query = "INSERT INTO question (question,
168             option_1, option_2, option_3, option_4,
169             correct_answer) VALUES (?, ?, ?, ?, ?, ?)";
170         pstmt = conn.prepareStatement(query);
171         pstmt.setString(1, questionText);
172         pstmt.setString(2, option1Text);
173         pstmt.setString(3, option2Text);
174         pstmt.setString(4, option3Text);
175         pstmt.setString(5, option4Text);
176         pstmt.setInt(6, correctAnswer);
177         pstmt.executeUpdate();
178
179         questionsList.add(new Question(questionText,
180             option1Text, option2Text, option3Text,
181             option4Text, correctAnswer));
182         loadQuestionsIntoTable();
183         JOptionPane.showMessageDialog(this, "Question
184             added successfully.");
185     } catch (Exception ex) {
186         JOptionPane.showMessageDialog(this, "Error
187             adding question: " + ex.getMessage());
188     }
189 }
190
191 private void editQuestion() {
192     int selectedRow = questionTable.getSelectedRow();
193     if (selectedRow == -1) {
194         JOptionPane.showMessageDialog(this, "Please select
195             a question to edit.");
196         return;
197     }
198
199     Question question = questionsList.get(selectedRow);
200
201     JTextField questionField = new JTextField(question.
202         getQuestionText());
203     JTextField option1Field = new JTextField(question.
204         getOption1());
205     JTextField option2Field = new JTextField(question.
206         getOption2());
207     JTextField option3Field = new JTextField(question.
208         getOption3());
209     JTextField option4Field = new JTextField(question.
210         getOption4());
211     JTextField correctAnswerField = new JTextField(String.
212         valueOf(question.getCorrectAnswer()));
213
214     Object[] fields = {
215         "Question:", questionField,
216         "Option 1:", option1Field,

```

```

204         "Option 2:", option2Field,
205         "Option 3:", option3Field,
206         "Option 4:", option4Field,
207         "Correct Answer (1-4):", correctAnswerField
208     };
209
210     int option = JOptionPane.showConfirmDialog(this, fields
211     , "Edit Question", JOptionPane.OK_CANCEL_OPTION);
212     if (option == JOptionPane.OK_OPTION) {
213         try {
214             String query = "UPDATE question SET question =
215             ?, option_1 = ?, option_2 = ?, option_3 = ?,
216             option_4 = ?, correct_answer = ? WHERE
217             question = ?";
218             pstmt = conn.prepareStatement(query);
219             pstmt.setString(1, questionField.getText());
220             pstmt.setString(2, option1Field.getText());
221             pstmt.setString(3, option2Field.getText());
222             pstmt.setString(4, option3Field.getText());
223             pstmt.setString(5, option4Field.getText());
224             pstmt.setInt(6, Integer.parseInt(
225                 correctAnswerField.getText()));
226             pstmt.setString(7, question.getQuestionText());
227             pstmt.executeUpdate();
228
229             question = new Question(questionField.getText()
230             , option1Field.getText(), option2Field.
231             getText(), option3Field.getText(),
232             option4Field.getText(), Integer.parseInt(
233                 correctAnswerField.getText()));
234             questionsList.set(selectedRow, question);
235             loadQuestionsIntoTable();
236             JOptionPane.showMessageDialog(this, "Question
237             updated successfully.");
238         } catch (Exception ex) {
239             JOptionPane.showMessageDialog(this, "Error
240             updating question: " + ex.getMessage());
241         }
242     }
243
244     private void deleteQuestion() {
245         int selectedRow = questionTable.getSelectedRow();
246         if (selectedRow == -1) {
247             JOptionPane.showMessageDialog(this, "Please select
248             a question to delete.");
249             return;
250         }
251
252         Question question = questionsList.get(selectedRow);
253
254         int option = JOptionPane.showConfirmDialog(this, "Are
255         you sure you want to delete this question?", "Delete
256         Question", JOptionPane.YES_NO_OPTION);
257         if (option == JOptionPane.YES_OPTION) {
258             try {
259                 String query = "DELETE FROM question WHERE
260                 question = ?";

```

```

247         pstmt = conn.prepareStatement(query);
248         pstmt.setString(1, question.getQuestionText());
249         pstmt.executeUpdate();
250
251         questionsList.remove(selectedRow);
252         loadQuestionsIntoTable();
253         JOptionPane.showMessageDialog(this, "Question
                deleted successfully.");
254     } catch (Exception ex) {
255         JOptionPane.showMessageDialog(this, "Error
                deleting question: " + ex.getMessage());
256     }
257 }
258 }
259
260 private void saveChanges() {
261     JOptionPane.showMessageDialog(this, "All changes saved
                successfully.");
262     new login().setVisible(true);
263     this.dispose();
264 }
265
266 public static void main(String[] args) {
267     SwingUtilities.invokeLater(new Runnable() {
268         public void run() {
269             new Questions_add();
270         }
271     });
272 }
273 }
274
275 class Question {
276     private String questionText;
277     private String option1;
278     private String option2;
279     private String option3;
280     private String option4;
281     private int correctAnswer;
282
283     public Question(String questionText, String option1, String
        option2, String option3, String option4, int
        correctAnswer) {
284         this.questionText = questionText;
285         this.option1 = option1;
286         this.option2 = option2;
287         this.option3 = option3;
288         this.option4 = option4;
289         this.correctAnswer = correctAnswer;
290     }
291
292     public String getQuestionText() {
293         return questionText;
294     }
295
296     public String getOption1() {
297         return option1;
298     }
299 }

```

```

300     public String getOption2() {
301         return option2;
302     }
303
304     public String getOption3() {
305         return option3;
306     }
307
308     public String getOption4() {
309         return option4;
310     }
311
312     public int getCorrectAnswer() {
313         return correctAnswer;
314     }
315 }

```

Listing 2.5: Question add

```

1     package quizzes;
2
3     import java.sql.*;
4     import java.awt.*;
5     import java.awt.event.*;
6     import javax.swing.*;
7     import java.util.ArrayList;
8     import java.util.List;
9
10    public class Questions extends JFrame implements ActionListener
11    {
12        private Connection conn;
13        private PreparedStatement pstmt;
14        private ResultSet rs;
15
16        private List<Question> questionsList;
17        private int currentQuestion = 0;
18        private int correctAnswersCount = 0;
19
20        private JLabel title;
21        private JLabel questionLabel;
22        private JRadioButton option1, option2, option3, option4;
23        private ButtonGroup options;
24        private JButton nextButton;
25        private JButton resultButton;
26
27        public Questions() {
28            connectToDatabase(); // Connect to the database
29            loadQuestionsFromDatabase(); // Load questions from the
30                                     database
31            openExamFrame(); // Open the exam frame
32        }
33
34        // Database connection method
35        public void connectToDatabase() {
36            try {
37                // Load MySQL driver
38                Class.forName("com.mysql.cj.jdbc.Driver");
39                conn = DriverManager.getConnection("jdbc:mysql://

```

```

localhost:3306/last?zeroDateTimeBehavior=
CONVERT_TO_NULL", "root", ""); // Update with
your database credentials
38 } catch (Exception e) {
39     JOptionPane.showMessageDialog(this, "Database
connection failed: " + e.getMessage());
40 }
41 }
42
43 // Load questions from the database
44 public void loadQuestionsFromDatabase() {
45     questionsList = new ArrayList<>();
46     try {
47         String query = "SELECT * FROM question";
48         pstmt = conn.prepareStatement(query);
49         rs = pstmt.executeQuery();
50         while (rs.next()) {
51             String questionText = rs.getString("question");
52             String option1Text = rs.getString("option_1");
53             String option2Text = rs.getString("option_2");
54             String option3Text = rs.getString("option_3");
55             String option4Text = rs.getString("option_4");
56             int correctAnswer = rs.getInt("correct_answer");
57             ;
58             questionsList.add(new Question(questionText,
option1Text, option2Text, option3Text,
option4Text, correctAnswer));
59         }
60     } catch (SQLException e) {
61         JOptionPane.showMessageDialog(this, "Error loading
questions from database: " + e.getMessage());
62     }
63 }
64
65 // Open the exam frame
66 public void openExamFrame() {
67     setTitle("E learning QuizPlat form");
68     setBounds(300, 90, 600, 400);
69     setDefaultCloseOperation(EXIT_ON_CLOSE);
70     setResizable(false);
71
72     Container container = getContentPane();
73     container.setLayout(null);
74
75     title = new JLabel("E learning QuizPlat form");
76     title.setFont(new Font("Arial", Font.PLAIN, 24));
77     title.setSize(300, 30);
78     title.setLocation(200, 30);
79     container.add(title);
80
81     questionLabel = new JLabel();
82     questionLabel.setFont(new Font("Arial", Font.PLAIN, 18)
);
83     questionLabel.setSize(550, 30);
84     questionLabel.setLocation(50, 80);
85     container.add(questionLabel);
86

```

```

87     option1 = new JRadioButton();
88     option1.setFont(new Font("Arial", Font.PLAIN, 15));
89     option1.setSize(500, 20);
90     option1.setLocation(50, 120);
91     container.add(option1);
92
93     option2 = new JRadioButton();
94     option2.setFont(new Font("Arial", Font.PLAIN, 15));
95     option2.setSize(500, 20);
96     option2.setLocation(50, 150);
97     container.add(option2);
98
99     option3 = new JRadioButton();
100    option3.setFont(new Font("Arial", Font.PLAIN, 15));
101    option3.setSize(500, 20);
102    option3.setLocation(50, 180);
103    container.add(option3);
104
105    option4 = new JRadioButton();
106    option4.setFont(new Font("Arial", Font.PLAIN, 15));
107    option4.setSize(500, 20);
108    option4.setLocation(50, 210);
109    container.add(option4);
110
111    options = new ButtonGroup();
112    options.add(option1);
113    options.add(option2);
114    options.add(option3);
115    options.add(option4);
116
117    nextButton = new JButton("Next");
118    nextButton.setFont(new Font("Arial", Font.PLAIN, 15));
119    nextButton.setSize(100, 30);
120    nextButton.setLocation(100, 300);
121    nextButton.addActionListener(this);
122    container.add(nextButton);
123
124    resultButton = new JButton("Show Result");
125    resultButton.setFont(new Font("Arial", Font.PLAIN, 15))
126    ;
127    resultButton.setSize(150, 30);
128    resultButton.setLocation(300, 300);
129    resultButton.addActionListener(this);
130    resultButton.setVisible(false);
131    container.add(resultButton);
132
133    loadQuestion(currentQuestion); // Load the first
134    question
135    setVisible(true);
136
137    }
138
139    // Load a specific question based on the index
140    public void loadQuestion(int questionIndex) {
141        if (questionIndex >= 0 && questionIndex < questionsList
142            .size()) {
143            Question question = questionsList.get(questionIndex
144            );
145            questionLabel.setText(question.getQuestionText());

```

```

141         option1.setText(question.getOption1());
142         option2.setText(question.getOption2());
143         option3.setText(question.getOption3());
144         option4.setText(question.getOption4());
145         options.clearSelection();
146     } else {
147         JOptionPane.showMessageDialog(this, "End of
148             questions reached!");
149     }
150 }
151
152 @Override
153 public void actionPerformed(ActionEvent e) {
154     if (e.getSource() == nextButton) {
155         // Check the current answer before moving to the
156         // next question
157         if (checkAnswer()) {
158             correctAnswersCount++; // Increment if the
159             // answer is correct
160         }
161         // Proceed to the next question
162         currentQuestion++;
163         if (currentQuestion < questionsList.size()) {
164             loadQuestion(currentQuestion);
165         } else {
166             // Hide next button and show result button
167             nextButton.setVisible(false);
168             resultButton.setVisible(true);
169         }
170     } else if (e.getSource() == resultButton) {
171         // Show the result with correct answers
172         StringBuilder resultMessage = new StringBuilder();
173         resultMessage.append("You got ").append(
174             correctAnswersCount).append(" out of ").append(
175             questionsList.size()).append(" correct answers.\n\n");
176
177         // Loop through all questions to show correct
178         // answers
179         for (int i = 0; i < questionsList.size(); i++) {
180             Question question = questionsList.get(i);
181             String userAnswer = getSelectedAnswer(i);
182             String correctAnswer = getCorrectAnswerText(
183                 question.getCorrectAnswer(), question);
184
185             resultMessage.append("Question ").append(i + 1)
186                 .append(": ").append(question.
187                     getQuestionText()).append("\n");
188             resultMessage.append("Your Answer: ").append(
189                 userAnswer).append("\n");
190             resultMessage.append("Correct Answer: ").append(
191                 correctAnswer).append("\n\n");
192         }
193
194         // Show the result message in a message dialog
195         JOptionPane.showMessageDialog(this, resultMessage.
196             toString(), "Quiz Results", JOptionPane.

```

```

186         INFORMATION_MESSAGE);
187
188         // After clicking OK, open the "Thank You" window
189         openThankYouWindow();
190     }
191
192     // Helper method to check if the selected answer is correct
193     private boolean checkAnswer() {
194         Question question = questionsList.get(currentQuestion);
195         int correctAnswerIndex = question.getCorrectAnswer();
196         if (correctAnswerIndex == 1 && option1.isSelected())
197             return true;
198         if (correctAnswerIndex == 2 && option2.isSelected())
199             return true;
200         if (correctAnswerIndex == 3 && option3.isSelected())
201             return true;
202         if (correctAnswerIndex == 4 && option4.isSelected())
203             return true;
204         return false;
205     }
206
207     // Helper method to get the selected answer for a question
208     private String getSelectedAnswer(int questionIndex) {
209         Question question = questionsList.get(questionIndex);
210         if (option1.isSelected()) return question.getOption1();
211         else if (option2.isSelected()) return question.
212             getOption2();
213         else if (option3.isSelected()) return question.
214             getOption3();
215         else if (option4.isSelected()) return question.
216             getOption4();
217         return "No answer"; // In case no option is selected
218     }
219
220     // Helper method to get the correct answer text by index
221     private String getCorrectAnswerText(int correctAnswerIndex,
222         Question question) {
223         switch (correctAnswerIndex) {
224             case 1: return question.getOption1();
225             case 2: return question.getOption2();
226             case 3: return question.getOption3();
227             case 4: return question.getOption4();
228             default: return "No answer";
229         }
230     }
231
232     // Method to open the "Thank You" window
233     private void openThankYouWindow() {
234         JFrame thankYouFrame = new JFrame("Thank You");
235         thankYouFrame.setSize(400, 200);
236         thankYouFrame.setLocationRelativeTo(this); // Center
237             the window on the screen
238         thankYouFrame.setDefaultCloseOperation(JFrame.
239             DISPOSE_ON_CLOSE);
240
241         JLabel thankYouLabel = new JLabel("Thank You! Join
242             Today", JLabel.CENTER);

```

```

232     thankYouLabel.setFont(new Font("Arial", Font.BOLD, 20))
233     ;
234     thankYouFrame.add(thankYouLabel);
235
236     thankYouFrame.setVisible(true);
237 }
238
239 public static void main(String[] args) {
240     SwingUtilities.invokeLater(new Runnable() {
241         public void run() {
242             new Questions();
243         }
244     });
245 }
246
247 // Question class to hold the question data
248 class Question {
249     private String questionText;
250     private String option1;
251     private String option2;
252     private String option3;
253     private String option4;
254     private int correctAnswer;
255
256     public Question(String questionText, String option1, String
        option2, String option3, String option4, int
        correctAnswer) {
257         this.questionText = questionText;
258         this.option1 = option1;
259         this.option2 = option2;
260         this.option3 = option3;
261         this.option4 = option4;
262         this.correctAnswer = correctAnswer;
263     }
264
265     public String getQuestionText() {
266         return questionText;
267     }
268
269     public String getOption1() {
270         return option1;
271     }
272
273     public String getOption2() {
274         return option2;
275     }
276
277     public String getOption3() {
278         return option3;
279     }
280
281     public String getOption4() {
282         return option4;
283     }
284
285     public int getCorrectAnswer() {
286         return correctAnswer;

```

```
287     }  
288 }
```

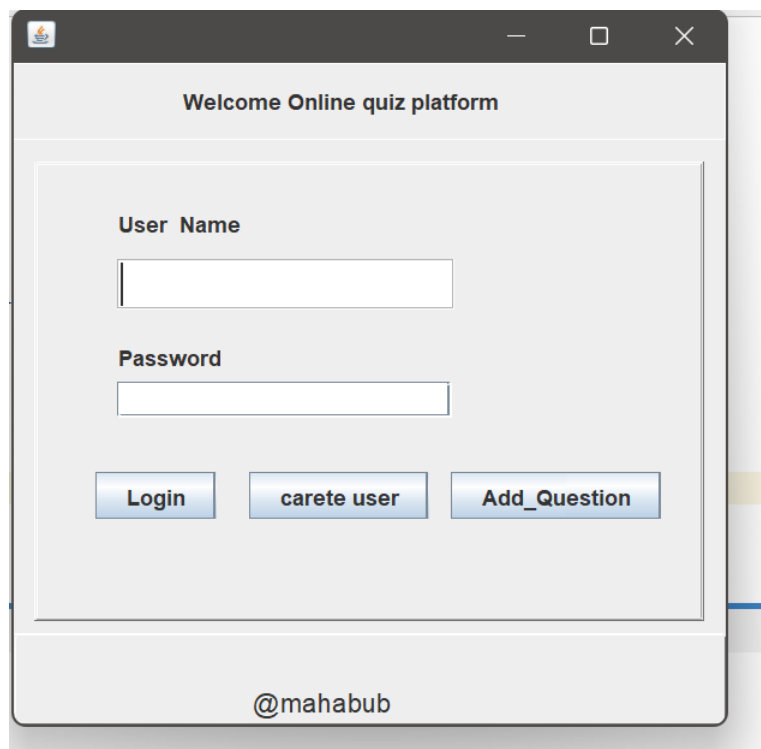
Listing 2.6: Question

Chapter 3

Performance Evaluation

3.1 Results Analysis

Result:



The screenshot shows a web application window titled "Welcome Online quiz platform". The interface includes a "User Name" label above a text input field, a "Password" label above another text input field, and three buttons: "Login", "carete user", and "Add_Question". The username field contains the text "@mahabub". The footer of the window displays "@mahabub".

Welcome Online quiz platform

User Name

carete user

Password

Login Add_Question

@mahabub

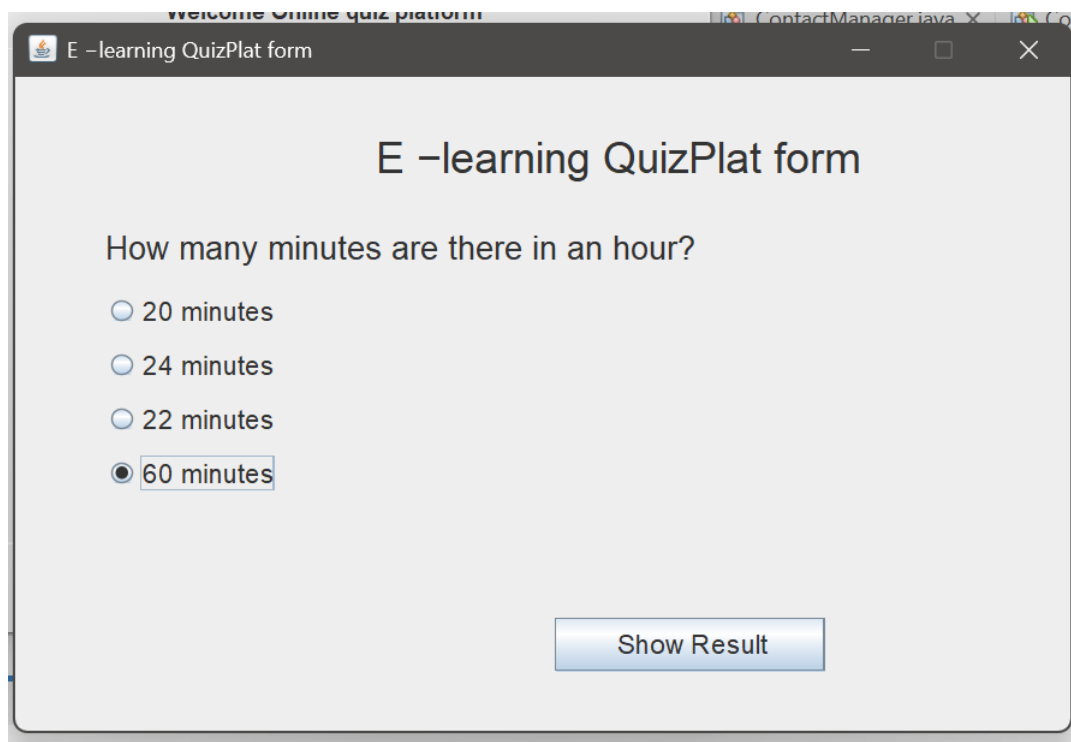
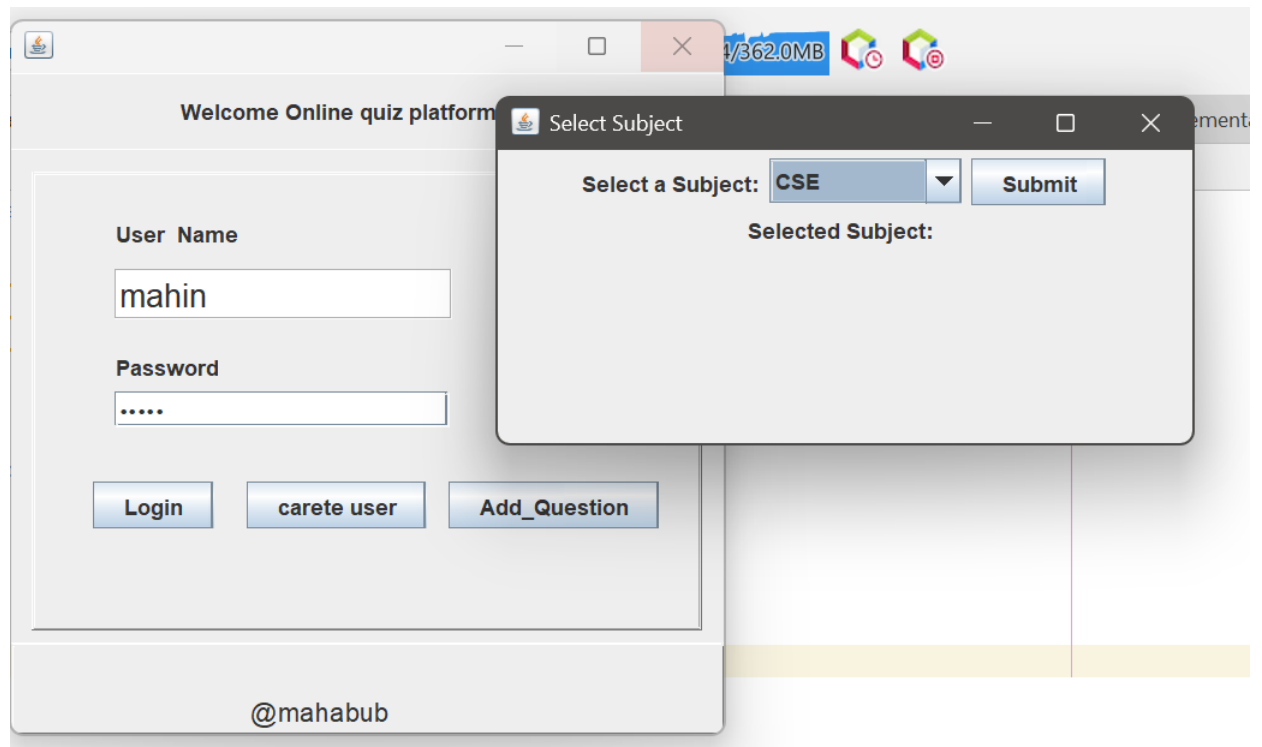
Name :

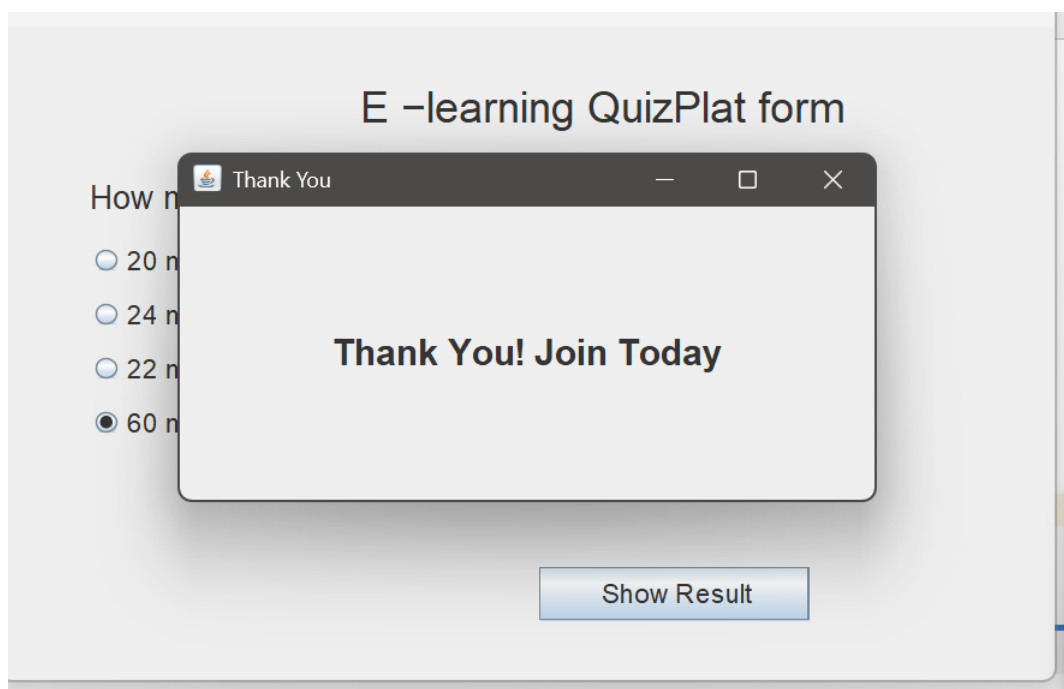
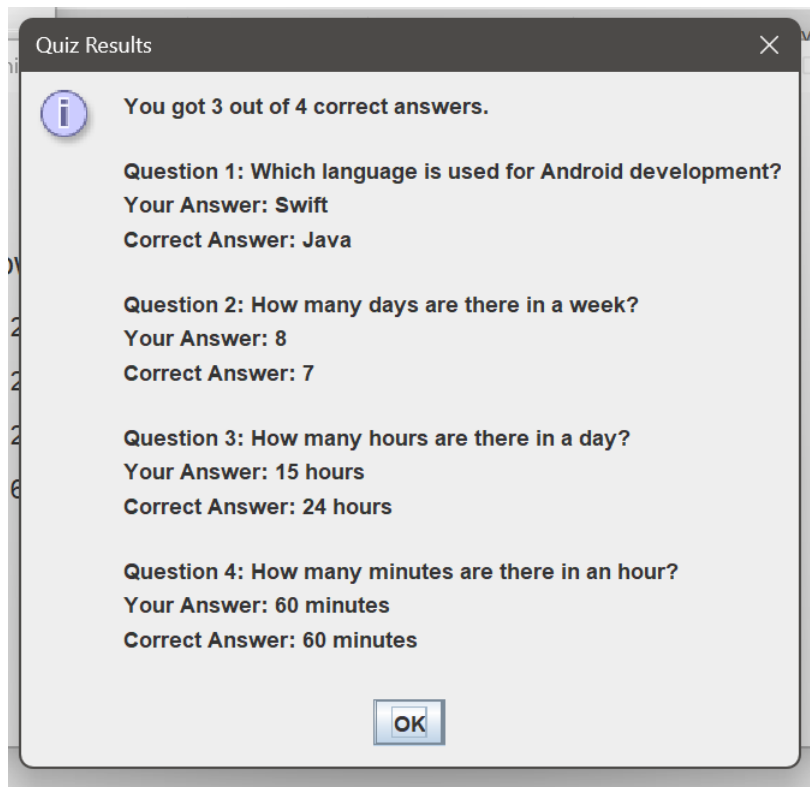
Password :

Name	Password
maju	maju
mahin	mahin

Question Management

Question	Option 1	Option 2	Option 3	Option 4	Correct Answer
Which language is u...	Python	Java	Kotlin	Swift	2
How many days are ...	5	6	7	8	3
How many hours are...	20 hours	24 hours	22 hours	15 hours	2
How many minutes a...	20 minutes	24 minutes	22 minutes	60 minutes	4





Chapter 4

Conclusion

4.1 Discussion

The E-learning Quiz Platform efficiently allows for assessment, while instant feedback and personalized insights are provided. Students can track their progress, identify strengths and weaknesses, and focus on areas needing improvement. Instructors get detailed analytics that allow them to refine quizzes for better learning outcomes. Interactive engagement is fostered with the platform's features for adaptive learning.

4.2 Limitations

- **Internet Dependency:** Requires a stable internet connection.
- **Security Risks:** Possibility of cheating or unauthorized access.
- **Limited Content:** Hard to create diverse question types.
- **Engagement Issues:** Students may lose interest without interactive features.
- **Technical Problems:** High traffic may cause slow performance.
- **Grading Accuracy:** Automated grading may not be perfect for essay-type questions.

4.3 Scope of Future Work

- **AI Integration:** Personalized learning paths based on performance.
- **Improved Analytics:** Enhanced data insights for better tracking.
- **Gamification:** Add rewards, badges, and leaderboards.
- **Mobile App:** Develop a mobile version for better accessibility.